

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 4

Queue dalam Python



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

Genap 2024/2025

A. Tujuan Praktikum

Agar setiap mahasiswa tahu cara mengaplikasikan struktur data Queue dalam python. Dan mahasiswa mampu memahami algoritma struktur data Queue, macam-macam penerapan, dan fungsi pada setiap linked list

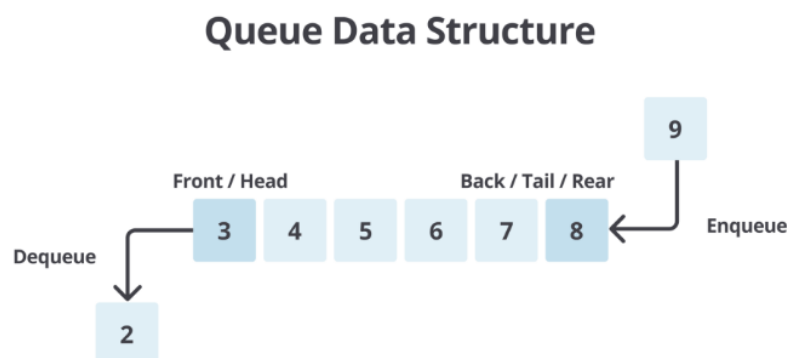
B. Dasar Teori

1. Pengertian Queue

Queue atau dalam Bahasa Indonesia yang berarti antrean adalah struktur data yang Menyusun elemen-elemen data dalam urutan linier. Prinsip dasar dari struktur data ini adalah “First in”, “First out” (FiFo) yang berarti elemen data yang pertama dimasukkan ke dalam antrean akan menjadi pertama pula untuk dikeluarkan.

2. Prinsip FiFo pada Queue

Cara kerja nya adalah seperti jejeran orang yang sedang menunggu antrean di supermarket dimana orang pertama yang datang adalah orang pertama pula yang keluar atau dilayani. Pada stuktur data ini, urutan pertama disebut Front atau head. Sebaliknya, pada urutan terakhir disebut Back, Rear, atau Tail. Proses untuk menambhakna data pada antrean disebut dengan Enqueue, sedangkan proses untuk menghapus data dari antrean disebut dengan Dequeue



Gambar 1.1

3. Fungsi Queue

Queue memiliki peran yang penting dalam berbagai aplikasi dan algoritma. Salah satu fungsi utamanya adalah mengatur dan mengelola antrian tugas atau operasi secara efisien. Dalam sistem komputasi, ia digunakan untuk menangani tugas-tugas seperti penjadwalan proses, antrian pesan, dan manajemen sumber daya.

4. Jenis-jenis Queue

Jenis struktur data ini dapat diklasifikasikan berdasarkan cara implementasinya, maupun berdasarkan penggunaannya.

1. Berdasarkan Implementasinya

- a. Linier/Simple Queue: Elemen-elemen data disusun dalam barisan linier dan penambahan serta penghapusan elemen hanya terjadi pada dua ujung barisan tertentu
- b. Circular Queue : Mirip dengan jenis linier, tetapi ujung-ujung barisan terhubung satu sama lain, menciptakan struktur antrian yang berputar

2. Berdasarkan Penggunaan

- a. Priority Queue : Setiap elemen memiliki prioritas tertentu. Elemen dengan prioritas tertinggi akan diambil terlebih dahulu
- b. Double-ended Queue (Deque) : Elemen dapat ditambahkan atau dihapus dari kedua ujung antrian

5. Keuntungan

- Data berjumlah besar dapat dikelola dengan mudah dan efisien
- Proses insert dan delete data dapat dilakukan dengan mudah karena mengikuti first in first out
- Efisien dalam menangani tugas berdasarkan urutan kedatangan

6. Kekurangan

- Tidak efisien untuk pencarian elemen tertentu dalam antrian
- Memerlukan alokasi memori yang cukup untuk menyimpan antrian

C. Hasil dan Pembahasan

1. Queue dengan List

```
#Queue dengan list
#Python program demosntrasi mengimplementasikan queue menggunakan
list

queue=[]

queue.append('n')
queue.append('o')
queue.append('f')
queue.append('a')
queue.append('l')

print('initial queue')
print('queue')

print('\nElements dequeued from queue :')
print(queue.pop(0))
print(queue.pop(0))
print(queue.pop(0))

print('\nQueue after removing elements')
print(queue)
```

Gambar 1.1

- Pertama membuat list kosong yang akan kita gunakan seperti gambar diatas kita membuat list dengan nama queue
- Menambahkan elemn ke dalam queue menggunaakn metode append
- Selanjutnya initianlisi queue
- Menghapus elemen queue menggunakan metode pop (0) artinya menghapus elemn dari awaln queue
- Hasilnya tiga elemn pertama n, o, f dihapus dari queue
- Queue akan berisi a dan l saja
- Kalau untuk pop kurang efisien untuk menghapus elemen karena memerlukan pergeseran elemen yang tersisa

2. Queue dengan Collections.deque

```
# Python program untuk mendemonstrasikan queue dengan collection.
deque
from collections import deque

queue = deque()

queue.append('n')
queue.append('o')
queue.append('f')
queue.append('a')
queue.append('l')

print('initial queue')
print(queue)

print('\nElements dequeued from queue :')
print(queue.popleft())
print(queue.popleft())
print(queue.popleft())

print('\nQueue after removing elements')
print(queue)
```

Gambar 1.2

- Pertama kita mengimport kelas deque dari modul collection
- Deque adalah struktur data yang mendukung operasi penghapusan efisien dari kedua ujungnya
- Selanjutnya membuat objek deque yang akan digunakan
- Menambahkan elemen ke dalam deque menggunakan metode append
- Inisialisasi deque akan berisi n,o,f,a,l
- Selanjutnya menghapus elemen dari deque menggunakan metode popleft
- Hasilnya 3 elemen pertama n,o,f akan dihapus dari queue
- Didalam queue akan menjadi a dan l saja
- Dengan deque operasi menghapus elemen dilakukan secara efisien karena tidak memerlukan pergeseran elemen yang tersisa

3. Queue dengan queue.Queue

```
from queue import Queue

qu = Queue(maxsize=5)

print(qu.qsize())

qu.put('n')
qu.put('o')
qu.put('p')
qu.put('a')
qu.put('l')

print("full:",qu.full())
print("size:",qu.qsize())

print('\nElements dequeued from queue :')
print(qu.get())
print(qu.get())
print(qu.get())

print("\nEmpty",qu.empty())
print("Full:",qu.full())
```

Gambar 1.3

- Pertama mengimpor class queue dari modul queue
- Membuat objek queue dengan batas maksimum sebesar 5
- Mencetak jumlah elemen dalam deque menggunakan qsize()
- Pada awalnya queue kosong sehingga hasilnya 0
- Selanjutnya menambahkan elemen ke dalam queue menggunakan metode put ()
- Hasilnya akan menjadi n,o,p,a,l
- Selanjutnya memberikan informasi apakah queue telah mencapai batas maksimum
- Untuk selanjutnya menghapus elemen dari queue menggunakan metode get ()
- Hasilnya tiga elemen pertama n,o,p akan dihapus dari queue
- Selanjutnya empty akan memberikan informasi apakah queue kosong setelah penghapusan elemen dan full untuk memberikan informasi apakah queue penuh setelah penghapusan elemen

4. Queue dengan Single Linked List

```
class Queue:
    def __init__(self):
        self.queue = list()

    def addtoqu(self, dataval):
        if dataval not in self.queue:
            self.queue.insert(0, dataval)
            return True
        return False

    def removefromq(self):
        if len(self.queue) > 0:
            return self.queue.pop()
        return "No elements in Queue!"

    def size(self):
        return len(self.queue)

TheQueue = Queue()
print(TheQueue.addtoqu("Nopal"))
print(TheQueue.addtoqu("Aji"))
print(TheQueue.addtoqu("Prasetyo"))
print(TheQueue.addtoqu("Ganteng"))
print(TheQueue.size())
print(TheQueue.removefromq())
print(TheQueue.removefromq())
print(TheQueue.size())
```

Gambar 1.4

- Yang pertama kita inisialisasi kelas nya, kelas queue memiliki attribute queue yang dibuat list
- Fungsi addtoqu digunakan untuk menambahkan elemen ke dalam queue, jika elemen belum ada dalam queue maka elemen tersebut dimasukan ke indeks 0 menggunakan insert (0, dataval). True jika operasi berhasil dan false jika elemen sudah ada dalam queue
- Selanjutnya removefromq digunakan untuk menghapus elemen dari queue. Jika queue kosong maka elemen dihapus dari ujung belakang queue menggunakan pop () dan jika queue kosong akan mengembalikan pesan no elements in Queue
- Fungsi size digunakan untuk mendapatkan jumlah elemen dalam queue menggunakan len(self.queue)
- Untuk objek The Queue dibuat kelas Queue
- Elemen-elemen ditambahkan ke dalam queue menggunakan metode addtoqu
- Untuk ukuran queue dicetak menggunakan metode size
- Elemen elemen dihapus dari queue menggunakan metode removefromq
- Untuk ukuran queue dicetak Kembali setelah penghapusan elemen.

D. Kesimpulan

Kesimpulan dari praktikum di atas adalah mengetahui konsep konsep dasar terkait struktur data Queue dan bagaimana cara menggunakan nya juga pada single linked yang kita pelajari sebelumnya

E. Referensi

- <https://www.geeksforgeeks.org/queue-data-structure/>